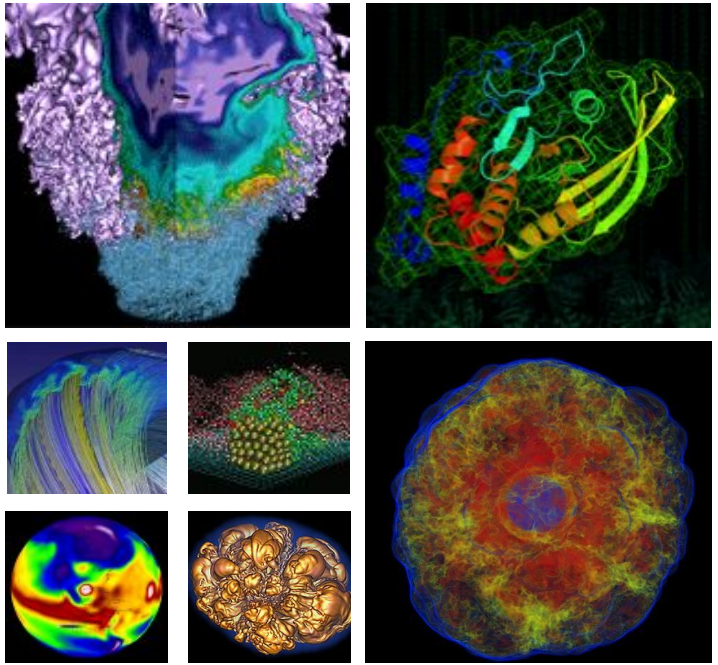
C++23 Intro: What's New and How to Use It



Weile Wei, Ph.D.

HPC Performance Engineer
Programming Environment and Model Group
National Energy Research Scientific Computing Center

WeileWei@lbl.gov | 06/20/2023

About this talk



What is C++23 and why do we need it?

- Many codes use older C++ versions
- We need modern, portable, safe, and performant C++

This talk is about:

- Understand some new features with easy-to-follow examples

This talk is <u>NOT</u> about:

- Deep dive into library implementation, proposals

Outline

- <stacktrace> - provides current execution context
- <expected> - error handling
- <mdspan> - multi-dimensional array
- views::zip_transform - zip multiple views, apply a function to generate a new view
- <stdfloat> - fixed-width floating point types

stacktrace intro

Quick question: How do you debug your C++ applications?

stacktrace intro

What is stacktrace?

- Provides stacktrace of the current execution context
- A container

Compilers support:

- gcc 12.1+, MSVC 19.34*
- Compiler flags needed for gcc: -std=c++23 -lstdc++_libbacktrace

Perlmutter@NERSC support:

- Gcc 12+ modules in Perlmutter are not yet enabled with `--enable-libstdcxx-backtrace=yes` upon configuration.
- Gcc 12 update is WIP.

stacktrace example 1

```cpp
#include <iostream>
#include <stacktrace>

int nested_func(int c) {
   std::cout << std::stacktrace::current() << '\n';
   return c + 1;
}
int func(int b) {
   return nested_func(b + 1);
}

int main(){
   std::cout << func(42);
}
```

Compiler Explorer:
https://godbolt.org/z/v3hKaEjeK

Expected output:

```
   0# nested_func(int) at /app/example.cpp:5
   1# func(int) at /app/example.cpp:9
   2#       at /app/example.cpp:13
   3#       at :0
   4#       at :0
   5#

44
```

stacktrace example 2

```cpp
#include <stacktrace>
#include <iostream>

void foo() {
   auto trace = std::stacktrace::current();
   for (const auto& entry: trace) {
      std::cout << "Description: " << entry.description() << std::endl;
      std::cout << "file: " << entry.source_file() << std::endl;
      std::cout << "line: " << entry.source_line() << std::endl;
      std::cout << "-----------------------------------" << std::endl;
   }
}

int main() {
   foo();
}
```

Compiler Explorer:
https://godbolt.org/z/Mrd73c8eW

Expected output:

```
Description: foo()
file: /app/example.cpp
line: 5
-----------------------------------
Description:
file: /app/example.cpp
line: 15
-----------------------------------
Description:
file:
line: 0
-----------------------------------
Description:
file:
line: 0
-----------------------------------
Description:
file:
line: 0
-----------------------------------
```

stacktrace reference

CPPreference:

- https://en.cppreference.com/w/cpp/header/stacktrace

Blog post:

- Sandor Dargo, C++23: The stacktrace library: https://www.sandordargo.com/blog/2022/09/21/cpp23-stacktrace-library

Video:

- Jason Turner, C++ Weekly - Ep 336 - C++23's Awesome std::stacktrace Library: https://youtu.be/9IcxniCxKlQ

expected intro

What is std::expected?

- a wrapper that contains either an expected or error value

Compilers support:

- Gcc 12, clang 16, MSVC 19.33

Perlmutter@NERSC support:

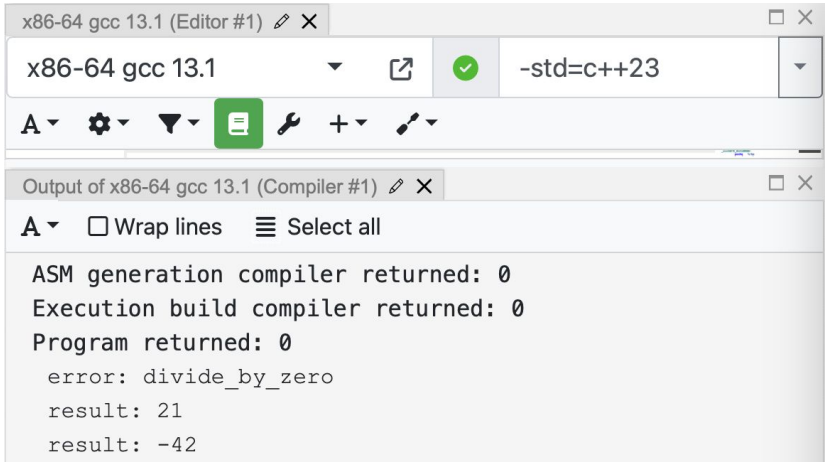
- gcc/12.1.0 and gcc/12.2.0 on Perlmutter both support <expected> with `-std=c++23` enabled during compilation

expected example

```
6  enum class errorCode {divide_by_zero, another_error};
7
8  std::expected<double, errorCode> safe_divide(double i, double j) {
9      if(j == 0.0) {
10         return std::unexpected(errorCode::divide_by_zero);
11     }
12     return i / j;
13 }
14
15 int main () {
16     auto process = [](const double& i, const double& j) {
17         auto res = safe_divide(i, j);
18         if (res.has_value()) {
19             std::cout << "result: " << *res << "\n";
20         } else if (res.error() == errorCode::divide_by_zero) {
21             std::cout << "error: divide_by_zero\n";
22         } else {
23             std::cout << "unpextced error!\n";
24         }
25     };
26
27     process(42.0, 0.0);
28     process(42.0, 2.0);
29     process(42.0, -1.0);
30 }
```

Compiler Explorer:
https://godbolt.org/z/oazaP54hf

Expected output:



expected reference

CPPreference:

- https://en.cppreference.com/w/cpp/utility/expected

std::mdspan intro

What is std::mdspan?

- Provides C++ multidimensional array support
- std::mdspan is a non-owning view into a contiguous sequence of objects that reinterprets it as a multidimensional array.

Compilers support:

- Clang 17 (supports std::extents only), nvhpc
- Use Kokkos mdspan implementation https://github.com/kokkos/mdspan

Perlmutter@NERSC support:

- nvhpc/23.1

From std::span to std::mdspan 2D

```cpp
#include <vector>
#include <span>
#include <https://raw.githubusercontent.com/kokkos/mdspan/single-header
#include <fmt/core.h>

int main()
{
    // Class template argument deduction (CTAD) (since C++17)
    std::vector v = {1,2,3,4};

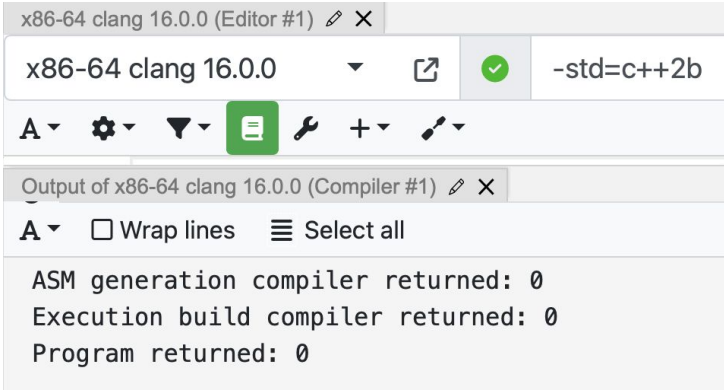
    // View v as contiguous memory representing 4 ints
    auto s = std::span(v.data(), 4);

    // View v as contiguous memory representing 2 rows
    // of 2 ints each.
    // Eventually, we will use std::mdspan provided by the standard
    auto ms = std::experimental::mdspan(v.data(),2,2);

    // access data via []operator
    for(size_t i=0; i<ms.extent(0); i++)
        for(size_t j=0; j<ms.extent(1); j++)
            ms[i, j] = 1000 + ms[i, j];
}
```

Compiler Explorer:
https://godbolt.org/z/6GPeoeeKM

Expected output:

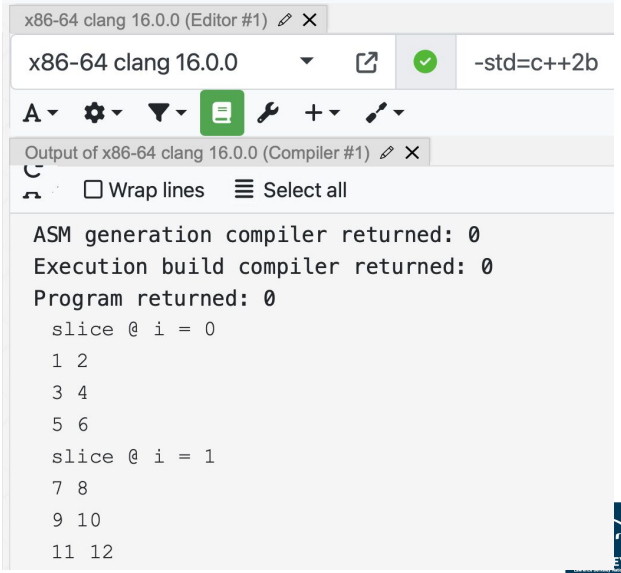


std::mdspan 3D example

```
5   int main()
6   {
7     std::vector v = {1,2,3,4,5,6,7,8,9,10,11,12};
8     // View the data as a 3D array 2 x 3 x 2
9     auto ms3 = std::experimental::mdspan(v.data(), 2, 3, 2);
10
11    // read back using 3D view
12    for(size_t i=0; i != ms3.extent(0); i++)
13    {
14      fmt::print("slice @ i = {}\n", i);
15      for(size_t j=0; j != ms3.extent(1); j++)
16      {
17        for(size_t k=0; k != ms3.extent(2); k++)
18          fmt::print("{} ",  ms3[i, j, k]);
19        fmt::print("\n");
20      }
21    }
22  }
```

Compiler Explorer:
https://godbolt.org/z/56ddP7rnE

Expected output:



std::mdspan::extents

What is extents?

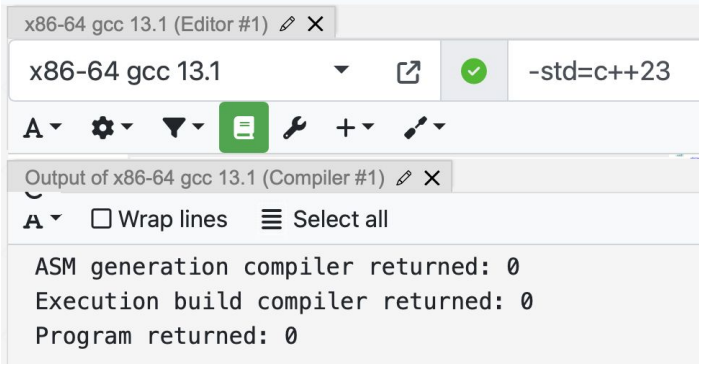
- A class template that expresses the shape of the mdspan.
- extents can either be dynamic, static, or a combination of both.

extents for mdspan example

```
9   std::vector v{1,2,3,4,5,6,7,8,9,10,11,12};
10
11  // An extents type using uint32_t to represent each extent,
12  // std::dynamic_extent means it's a dimension given at runtime
13  using ext_t1 = stdex::extents<uint32_t,
14          std::dynamic_extent, std::dynamic_extent>;
15  int rows = 3, cols = 4;
16  auto ms = stdex::mdspan<int, ext_t1>(v.data(), rows, cols);
17
18  assert(ms.extent(0) == rows);
19  assert(ms.extent(1) == cols);
20  static_assert(decltype(ms)::rank() == 2);
21
22  // runtime and compile-time dimensions can be mixed
23  constexpr int d1 =2, d2 =3;
24  int d3 = 2;
25  using ext_t2 = stdex::extents<uint32_t,
26          d1, d2, std::dynamic_extent>;
27
28  auto msd = stdex::mdspan<int, ext_t2>(v.data(), d3);
29
30  static_assert(msd.extent(0) == d1);
31  static_assert(msd.extent(1) == d2);
32  assert(msd.extent(2) == d3);
33  static_assert(decltype(msd)::rank() == 3);
```

Compiler Explorer:
https://godbolt.org/z/ovvhzPGKn

Expected output:



mdspan on Perlmutter

```cpp
#include <iostream>
#include <experimental/mdspan>
#include <vector>

namespace stdex = std::experimental;

int main()
{
  std::vector<int> v = {1,2,3,4,5,6,7,8,9,10,11,12};
  // View data as contiguous memory
  // representing 2 rows of 6 ints each
  constexpr int rows = 2;
  using extents_t = stdex::extents<int,
    rows, stdex::dynamic_extent>;
  auto ms2 = stdex::mdspan<int,
    extents_t, stdex::layout_left> (v.data(), 6);

  // write data using 2D view
  for(size_t i=0; i != ms2.extent(0); i++) {
    for(size_t j=0; j != ms2.extent(1); j++) {
      std::cout << ms2(i, j) << " ";
    }
  }
  std::cout << "\n";
```

Source code:

https://godbolt.org/z/o5Yhcd76o

On Perlmutter Today:

user@login01:~/src/test-mdspan> module load nvhpc/23.1

user@login01:~/src/test-mdspan> nvc++ --experimental-stdpar mdspan.cpp -o mdspan

user@login01:~/src/test-mdspan> ./mdspan
1 3 5 7 9 11 2 4 6 8 10 12

Difference between kokkos & nvhpc compiler implementations:

- Nvhpc mdspan does not support CTAD (Class template argument deduction).
- Nvhpc mdspan implementation uses `()operator` to access the data instead of multi-parameter `[]operator` - a feature which was voted into C++23.

std::mdspan customization

```
template<class T,
         class Extents,
         class LayoutPolicy = std::layout_right,
         class AccessorPolicy = std::default_accessor<T>>
class std::mdspan;
```

The `LayoutPolicy` defines the data layout:

- Default rightmost extent is contiguous, aka, row major, like C++, Numpy.
- Or std::layout_left, aka, column major, like Fortran and MATLAB.

The `AccessorPolicy` defines how the array stores its elements, and how to use the offset from the `LayoutPolicy` to get a reference to an element of the array.

- Default `AccessorPolicy` takes a pointer p and an offset i, and returns p[i].

std::mdspan abstraction

void your_function (std::mdspan<T, Extents, Layout, Accessor> m);

your_function(Eigen::Matrix<double, Eigen::Dynamic, Eigen::Dynamic> (.);

your_function(boost::numeric::ublas::matrix<double> (...);

your_function (Mat {.}); // PETSC

your_function(blaze::DynamicMatrix‹double, blaze::rowMajor›{.});
your_function(cutlass::HostTensor‹float, cutlass::layout::ColumnMajor›{]);

std::mdspan reference

CPPreference:

https://en.cppreference.com/w/cpp/language/class_template_argument_deduction

https://en.cppreference.com/w/cpp/container/mdspan

Kokkos documentation:

https://github.com/kokkos/mdspan/wiki/A-Gentle-Introduction-to-mdspan

Multidimensional C++ - Bryce Adelstein Lelbach - CppNorth 2022

https://youtu.be/aFCLmQEkPUw

zip_transform intro

What is zip_transform?

- zip several views, apply a transformation function, generate a new view

Compilers support:

- Gcc 13, clang 15 (partial), MSVC 19.33 (partial)

Perlmutter@NERSC support:

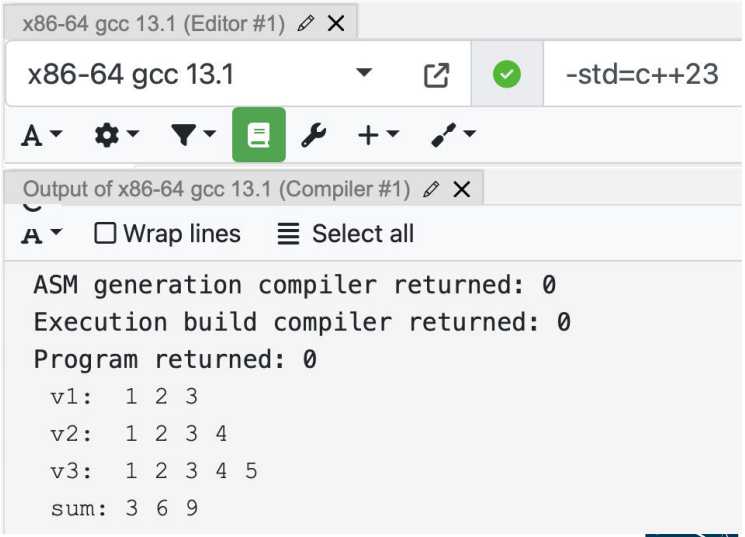
- Gcc 13 module is WIP.

zip_transform example

```
15  int main() {
16      auto v1 = std::vector<float>{1, 2, 3};
17      auto v2 = std::list<short>{1, 2, 3, 4};
18      auto v3 = std::to_array({1, 2, 3, 4, 5});
19      auto add = [](auto a, auto b, auto c) { return a + b + c; };
20      auto sum = std::views::zip_transform(add, v1, v2, v3);
21      print("v1:  ", v1);     // 1 2 3
22      print("v2:  ", v2);     // 1 2 3 4
23      print("v3:  ", v3);     // 1 2 3 4 5
24      print("sum: ", sum);    // 3 6 9
25  }
26
```

Compiler Explorer:
https://godbolt.org/z/e9e7xWsfT

Expected output:



zip_transform reference

CPPreference:

- https://en.cppreference.com/w/cpp/ranges/zip_transform_view
- https://en.cppreference.com/w/cpp/ranges (std::ranges – C++20)

stdfloat intro

What is stdfloat?

- part of the type support library, providing fixed width floating-point types.

Compilers support:

- Gcc 13, MSVC 19.37

Perlmutter@NERSC support:

- Gcc 13 module WIP.

Fixed width floating-point types

Fixed width floating-point types (since C++23)

If the implementation supports any of the following ISO 60559 types, then:

- the corresponding macro is predefined,
- the corresponding floating-point literal suffix is available, and
- the corresponding type alias name is provided:

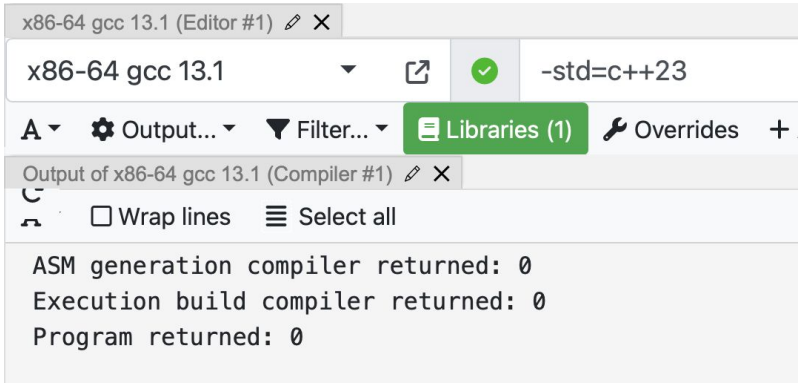
Type name Defined in header <stdfloat>	**Literal suffix**	**Predefined macro**	**C language type**	**Type properties**			
				bits of storage	bits of precision	bits of exponent	max exponent
std::float16_t	f16 or F16	__STDCPP_FLOAT16_T__	_Float16	16	11	5	15
std::float32_t	f32 or F32	__STDCPP_FLOAT32_T__	_Float32	32	24	8	127
std::float64_t	f64 or F64	__STDCPP_FLOAT64_T__	_Float64	64	53	11	1023
std::float128_t	f128 or F128	__STDCPP_FLOAT128_T__	_Float128	128	113	15	16383
std::bfloat16_t	bf16 or BF16	__STDCPP_BFLOAT16_T__	(N/A)	16	8	8	127

stdfloat example

```
1  #include <stdfloat>
2
3  int main()
4  {
5      std::float64_t f = 0.1f64;
6  }
7
```

Compiler Explorer:
https://godbolt.org/z/MP4MM4Tc9

Expected output:



stdfloat reference

CPPreference:

- https://en.cppreference.com/w/cpp/types/floating-point

Summary

- <stacktrace> - print current execution context
- <expected> - error handling
- <mdspan> - multi-dimensional array & extents
- views::zip_transform - zip and transform multiple containers, and generate a new view
- <stdfloat> - fixed-width floating pointing types

C++23

The next generation of the C++ standard.

See: The current IS schedule for C++23 (2023-05-10).

This section is incomplete

New language features

New library features

New modules

- `std` and `std.compat`

New headers

- `<expected>`
- `<flat_map>`
- `<flat_set>`
- `<generator>`
- `<mdspan>`
- `<print>`
- `<spanstream>`
- `<stacktrace>`
- `<stdfloat>`

C compatibility headers:

- `<stdatomic.h>`

Future C++ talks & Collaborations

Next talk:

- Asynchronous and Parallel programming with Sender/Receiver (C++ 26)
  - Target timeframe: July ~ August
  - Presenters: Weile Wei (NERSC) and Muhammad Haseeb (NERSC)

If you …

- Excited about learning other modern C++ features?!
- Would like to collaborate?
- Would like to request new compiler features?
- Have any feedbacks?
  - Feel free to contact me: WeileWei@lbl.gov :)





Thank You

New header <stdatomic.h>

This header was originally in the C standard library.

This header is part of the concurrency support library.

It is unspecified whether <stdatomic.h> provides any declarations in namespace std.

Macros

`_Atomic` (C++23)	compatibility macro such that `_Atomic(T)` is identical to `std::atomic<T>` (function macro)
`ATOMIC_FLAG_INIT` (C++11)	initializes an `std::atomic_flag` to `false` (macro constant)

Types

`atomic_flag` (C++11)	the lock-free boolean atomic type (class)
`memory_order` (C++11)	defines memory ordering constraints for the given atomic operation (enum)
`atomic_bool` (C++11)	`std::atomic<bool>` (typedef)
`atomic_char` (C++11)	`std::atomic<char>` (typedef)
`atomic_schar` (C++11)	`std::atomic<signed char>` (typedef)
`atomic_uchar` (C++11)	`std::atomic<unsigned char>` (typedef)
`atomic_short` (C++11)	`std::atomic<short>` (typedef)
`atomic_ushort` (C++11)	`std::atomic<unsigned short>` (typedef)
`atomic_int` (C++11)	`std::atomic<int>` (typedef)

backup

backup

extent for std::span example

```cpp
#include <span>
#include <iostream>
#include <vector>

int main()
{
    std::vector v {0, 1, 2, 3, 4, 5 };

    // dynamic extent
    std::span<int> vs1(v);
    for(auto e: vs1) {
        std::cout << e << " ";
    }
    std::cout << "\n";

    constexpr int extend_sz = 3;
    // compile-time extent
    std::span<int, extend_sz> vs2(v);
    for(auto e: vs2) {
        std::cout << e << " ";
    }
    std::cout << "\n";
}
```

Compiler Explorer:
https://godbolt.org/z/aGvK5YGYc

Expected output:

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1 -std=c++23

Output of x86-64 gcc 13.1 (Compiler #1)

Wrap lines Select all

```
ASM generation compiler returned: 0
Execution build compiler returned: 0
Program returned: 0
0 1 2 3 4 5
0 1 2
```

flat_map, flat_set

stacktrace

stacktrace is an alias for basic_stacktrace with the default allocator. We can use custom allocator to construct basic_stacktrace.

std::stacktrace( ) is just a constructor call and it instantiates a new basic_stacktrace container. If you want to get the stacktrace of the current execution context, call the current static member function instead of the constructor.

ranges intro (C++20)

What is ranges?

-

Compilers support:

-

Ranges zip_view

zip_view:

Expected output: